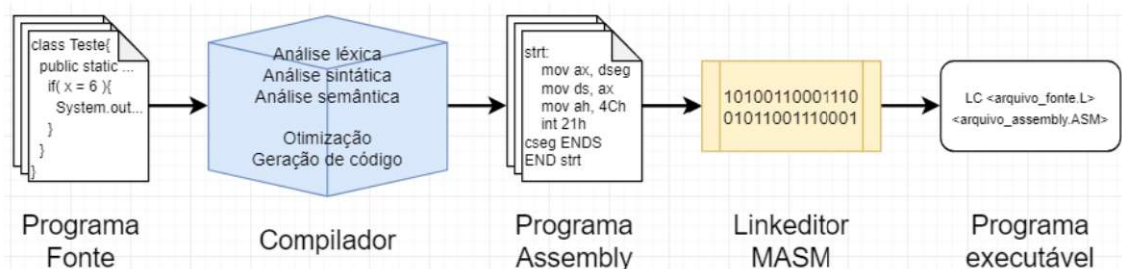

Trabalho Prático – Entrega Final 28/11/2025

Desenvolvimento de um compilador em linguagem imperativa simplificada**I – Objetivo**

O objetivo do trabalho prático é desenvolver um compilador completo que seja capaz de traduzir programas escritos na linguagem fonte “LC” para um subconjunto de instruções da linguagem Assembly (80x86). Tanto a linguagem fonte quanto a geração do assembler deverão ser realizadas durante o semestre. Para o final do TP, o compilador desenvolvido deve produzir um arquivo texto com extensão ASM, o qual será convertido em linguagem de máquina pelo montador MASM e executado via linha de comando. Em caso de detecção de erros no programa fonte, o compilador deve reportar o erro encontrado e terminar o processo de compilação.

As mensagens de erros deverão seguir as especificações do formato rigorosamente.

O programa executável do compilador desenvolvido deve ter o nome “LC”, e também deve ter dois parâmetros, recebendo os argumentos por linha de comando: 1) nome completo do programa fonte a ser compilado com extensão .LC; e 2) nome completo do programa assembly com extensão .ASM a ser gerado.

**II – Definição da linguagem fonte do compilador**

O compilador desenvolvido utiliza uma linguagem imperativa simplificada, que possui características das linguagens C e Pascal. A linguagem trata 4 tipos básicos de variáveis:

int	O tipo <i>int</i> é um escalar que varia de -32768 a 32767, ocupando 2 bytes.
byte	O tipo <i>byte</i> é um escalar que varia de 0 a 255, podendo ser escrito em formato decimal ou hexadecimal. As constantes em formato hexadecimal são no formato 0hHH, onde HH é um número hexadecimal (0~9) ('A'~'F'), sendo que as letras de A até F são maiúsculas.
string	O tipo <i>string</i> é um arranjo que pode conter até 255 caracteres úteis e quando armazenado em memória, é finalizado pelo caracter '\$'. Variáveis do tipo <i>string</i> ocupam 256 bytes de memória.
boolean	O tipo <i>boolean</i> pode ter os valores TRUE e FALSE, ocupando um byte de memória (0h para falso e FFh para verdadeiro).

No arquivo fonte, os caracteres permitidos são: letras, dígitos, espaço, sublinhado, ponto, vírgula, ponto-e-vírgula, dois-pontos, parênteses, colchetes, chaves, mais, menos, aspas, apóstrofo, barra, barra invertida, barra em pé, exclamação, interrogação, maior, menor e igual, além da quebra de linha (bytes 0Dh e 0Ah).

ATENÇÃO: Qualquer outro caractere é considerado inválido.

Variáveis do tipo *string* são delimitadas por aspas e não podem conter quebra de linha.

Os identificadores de constantes e variáveis são compostos de letras, dígitos e o sublinhado, começando necessariamente por uma letra ou sublinhado e podem ter no máximo 255 caracteres. A linguagem deve fazer distinção entre maiúsculas e minúsculas (*sensitive-case*).

As palavras a seguir são reservadas:

final	int	byte	string	while	if	else
and	or	not	==	=	(	)
<	>	<>	>=	<=	,	+
-	*	/	;	begin	end	readln
write	writeln	true	false	boolean		

Os comandos permitem atribuição para variáveis, entrada de valores pelo teclado e saída de valores para a tela, blocos (*begin - end*), estruturas de repetição (*while*), estruturas de teste (*if - else*), expressões aritméticas com inteiros e bytes, expressões lógicas e relacionais, além de atribuição, concatenação e comparação de igualdade entre strings. A ordem de precedência para as expressões deve ser:

- parênteses;
- negação lógica (*not*);
- multiplicação aritmética (*), lógica (*and*) e divisão (/);
- subtração (-), adição aritmética (+), lógica (*or*) e concatenação de strings (+);
- comparação aritmética (==, <>, <, >, <=, >=) e entre strings (==).

Comentários vêm entre /* */ ou entre { }.

A quebra de linha e o espaço devem ser usados como delimitadores de lexemas.

A estrutura básica do programa fonte deve seguir a seguinte estrutura:

Declarações e Bloco

Siga, atentamente, a descrição informal da sintaxe das declarações e comandos da linguagem a ser desenvolvida:

- Declaração de variáveis: **tipo lista-de-ids**;
onde *tipo* pode ser *int*, *boolean*, *byte* ou *string* e *lista-de-ids* é uma série de 1 ou mais identificadores, separados por vírgulas. As variáveis podem ser opcionalmente

inicializadas na forma: *id = valor*; onde *id* é um identificador e *valor* uma constante decimal, precedida ou não de sinal negativo, hexadecimal, lógica ou do tipo *string*.

2. Declaração de constantes: ***final id = valor***;
onde *id* é um identificador e *valor* é uma constante numérica, precedida ou não de sinal negativo, hexadecimal, lógica ou do tipo *string*.
3. Blocos são da forma: ***begin Bloco de Comandos end***
Dentro do bloco de comandos podem haver nenhum ou vários comandos em qualquer ordem.
4. Comando de atribuição: ***id = expressão***;
5. Comando de repetição:

while *expressão comando*
while *expressão bloco*
onde *expressão* é do tipo lógico.

6. Comando de teste:

if *expressão comando1*
if *expressão comando1 else comando2*
onde *expressão* é do tipo lógico, e *comando1* e *comando2* podem ser substituídos por blocos.

7. Comando nulo: ;
A linha contém apenas pelo sinal *ponto-vírgula*, e nada é executado neste comando.
8. Comando de leitura: ***readln, id***;
onde *id* é um identificador de variável inteira, byte ou string.

9. Comandos de escrita:

write, *lista_expressões*;
writeln, *lista_expressões*;

onde *lista_expressões* é uma lista de uma ou mais expressões numéricas ou do tipo *string*, separadas por vírgulas. O comando *writeln*, quando executado, causa a quebra de linha após a impressão.

III – Considerações gerais para todas as práticas:

1. O trabalho deverá ser feito em grupo, previamente definido, sem qualquer participação de outros grupos e/ou ajuda de terceiros. Cada aluno deve participar ativamente em todas as etapas do TP. Após serem designados os componentes dos

grupos, estes não poderão ser alterados durante o semestre. O(A)s aluno(a)s que não tiverem feito grupos até o começo das implementações, ficarão sem nota do TP.

2. A codificação do trabalho deve ser feita na linguagem Java de programação, em uma plataforma de desenvolvimento homologada e instalada nos laboratórios de Informática. Não poderão ser utilizados bibliotecas gráficas ou qualquer recurso que não esteja previamente definido na descrição do TP pelo professor.
3. Os trabalhos devem ser postados na forma de um arquivo compactado no formato ZIP ou RAR, e deve ter o nome ou RA dos integrantes do grupo separados por sublinhado, por exemplo: *nome1_nome2_nome3_nome4.zip*. Obviamente, os códigos fontes devem estar no arquivo compactado, e também, devem conter o nome de todos os componentes do grupo nas primeiras linhas do arquivo principal.
4. A cópia de TPs, códigos na sua totalidade ou parcialmente copiados, “encomendados” ou qualquer outra barbaridade do gênero, serão zerados para todos os integrantes dos grupos envolvidos, independente de qual grupo lesou ou foi lesado. É de responsabilidade do grupo manter o sigilo sobre a sua codificação, evitando que outros grupos tenham acesso a ele.
5. Será pedida ao Colegiado uma advertência formal no caso de cópia por má fé.
6. Para a apresentação do TP, poderão ser feitas perguntas relativas ao trabalho a qualquer integrante do grupo. Assim, todos os integrantes devem comparecer e serem capazes de responder a quaisquer perguntas e/ou alterar o código de qualquer parte do trabalho a pedido do professor, a fim de avaliar individualmente.
7. **ATENÇÃO:** A especificação do trabalho seja rigorosamente obedecida, principalmente com relação à interface com o usuário, uma vez que parte da correção será automatizada. Observe atentamente o nome do programa executável, os seus argumentos de entrada e formatos das mensagens. Os trabalhos com interfaces diferentes das especificadas correm o risco de não serem avaliadas.
8. A avaliação será baseada nos seguintes critérios:
 - Correção e robustez dos programas.
 - Conformidade às especificações prévias.
 - Clareza de codificação (comentários, indentação, nomes dos identificadores).
 - Organização dos arquivos no projeto o compilador.
 - Parametrizações dos métodos.
 - Apresentação individual dos integrantes.
 - Documentação do compilador desenvolvido.

Bom trabalho!